

Módulo: O Modelo Relacional — Conceitos Fundamentais

Curso: Banco de Dados Relacionais aplicado à Administração Tributária **Público-alvo:** Profissionais da Secretaria de Estado da Fazenda da Paraíba (SEFAZ-PB)
SGBD utilizado: Microsoft SQL Server (T-SQL) — ferramenta: SQL Server Management Studio (SSMS) **Carga horária sugerida:** 8 horas (5h teoria + 3h prática)
Base teórica: conceitos clássicos do Modelo Relacional (Codd, 1970), na linha adotada por ELMASRI & NAVATHE, *Fundamentals of Database Systems*

Sumário

1. [Objetivos do Módulo](#)
 2. [Contexto Histórico do Modelo Relacional](#)
 3. [Conceitos Fundamentais: Relação, Tupla, Atributo e Domínio](#)
 4. [Esquema de Relação × Instância \(Snapshot\)](#)
 5. [Grau da Relação e Propriedades Formais](#)
 6. [Chaves de uma Relação](#)
 7. [Chave Estrangeira e Autorrelacionamento](#)
 8. [Regras de Integridade](#)
 9. [Operações de Atualização e Violações de Integridade](#)
 10. [Álgebra Relacional — Fundamentos e Operadores Básicos](#)
 11. [Operadores Derivados: Junção, Interseção e Divisão](#)
 12. [Roteiro Prático — Do Conceito Formal ao T-SQL](#)
 13. [Exercícios Práticos](#)
 14. [Glossário](#)
 15. [Referências](#)
-

1. Objetivos do Módulo

Ao final deste módulo, o participante será capaz de:

- Situar o Modelo Relacional em seu contexto histórico e explicar por que ele se tornou o padrão dominante em SGBDs comerciais.
 - Definir com rigor os conceitos de **relação**, **tupla**, **atributo**, **domínio**, **grau** e **instância (snapshot)**.
 - Explicar as propriedades formais que distinguem uma relação matemática de uma tabela SQL comum.
 - Diferenciar **superchave**, **chave candidata**, **chave primária** e **chave estrangeira**, inclusive em **autorrelacionamentos**.
 - Enunciar e aplicar as três regras de integridade do modelo relacional: **integridade de chave**, **integridade de entidade (identidade)** e **integridade referencial**.
 - Analisar as consequências de operações de inserção, remoção e atualização sobre a integridade dos dados, e as estratégias possíveis diante de uma violação.
 - Compreender a **álgebra relacional** como fundamento formal das consultas SQL, aplicando os operadores de seleção, projeção, produto cartesiano, união, diferença, junção, interseção e divisão.
 - Traduzir cada operador da álgebra relacional para sua construção equivalente em T-SQL.
-

2. Contexto Histórico do Modelo Relacional

O Modelo Relacional foi proposto em 1970 pelo matemático e pesquisador **Edgar F. Codd**, então na IBM, em um artigo que revolucionou a forma de organizar dados em sistemas computacionais. Antes de Codd, os bancos de dados dominantes eram os modelos **hierárquico** e **em rede**, que exigiam que o programador conhecesse antecipadamente os caminhos de acesso físico aos dados (ponteiros entre registros). Codd propôs, em vez disso, representar os dados como **relações matemáticas** — na prática, tabelas — manipuláveis por meio de uma linguagem declarativa, sem que o usuário precisasse saber *como* os dados estavam fisicamente organizados.

Por que o modelo se popularizou:

- **Simplicidade conceitual:** dados organizados em tabelas são intuitivos, inclusive para usuários não técnicos (por exemplo, um auditor fiscal consegue entender uma tabela de "Autos de Infração" sem conhecer detalhes de implementação).

- **Independência de dados:** a estrutura lógica (tabelas, colunas) é separada da estrutura física de armazenamento, permitindo otimizações internas do SGBD sem afetar as consultas.
- **Base matemática sólida:** por se apoiar na teoria de conjuntos e na lógica de predicados, o modelo permite provar formalmente propriedades das consultas (a **álgebra relacional**, tratada na seção 10).
- **Desempenho competitivo:** os primeiros protótipos (como o **System R**, da IBM) demonstraram que era possível implementar o modelo com desempenho aceitável para aplicações comerciais reais.

O **System R** foi um dos primeiros SGBDs a implementar esses conceitos na prática, e a partir dele — e de trabalhos correlatos — surgiram sistemas como **DB2** (IBM), **Oracle**, **Informix**, **Ingres**, **Sybase**, e, mais tarde, o próprio **Microsoft SQL Server**, todos seguindo a mesma base teórica relacional que usaremos ao longo deste curso.

Por que isso importa para a SEFAZ-PB: todo o ecossistema de sistemas fiscais brasileiros — desde o cadastro de contribuintes até os sistemas que recebem NF-e, CT-e e EFD — é construído sobre SGBDs relacionais. Entender os fundamentos do modelo relacional é entender a lógica que rege *qualquer* consulta, relatório ou cruzamento de dados que a SEFAZ-PB venha a produzir.

3. Conceitos Fundamentais: Relação, Tupla, Atributo e Domínio

O Modelo Relacional representa os dados de um banco de dados como uma **coleção de relações** — na prática, um conjunto de tabelas.

3.1 Relação (Relation)

Uma **relação** corresponde a uma tabela. Toda relação tem um **nome único** dentro do banco de dados e um conjunto de **atributos** (colunas), cada um com seu próprio nome e domínio.

Exemplo fiscal — a relação AuditorFiscal:

Matrícula	Nome	Endereço	Função	Salário	Unidade
100	Ana Ribeiro	R. Pedro I, 12, Alto Branco	Auditor Tributário	12.500,00	UF1
250	Pedro Souza	R. J. Silva, 24, Liberdade	Auditor Fiscal Chefe	18.900,00	UF1
108	André Lima	R. Itália, 33, Bairro das Nações	Técnico Tributário	8.700,00	UF2
210	Paula Nogueira	R. Pará, 98, Bairro dos Estados	Auditora Fiscal Chefe	19.200,00	UF2

Cada linha desta tabela representa um auditor fiscal distinto. Cada coluna representa uma característica (atributo) que se aplica a todos os auditores.

3.2 Tupla (Tuple)

Uma **tupla** é uma linha da relação — um conjunto de valores, um para cada atributo da relação.

Exemplo: a tupla (100, 'Ana Ribeiro', 'R. Pedro I, 12, Alto Branco', 'Auditor Tributário', 12500.00, 'UF1') é uma ocorrência específica da relação AuditorFiscal.

3.3 Atributo (Attribute)

Um **atributo** é uma coluna da relação: um nome que representa um "papel" desempenhado por um domínio dentro daquela relação. Na tabela acima, Matrícula, Nome, Endereço, Função, Salário e Unidade são os cinco atributos da relação AuditorFiscal.

Detalhe importante: um mesmo domínio pode desempenhar papéis diferentes — e portanto ter nomes de atributo diferentes — em relações distintas. Por exemplo, o domínio "texto de até 6 caracteres representando um código de unidade" pode aparecer como o atributo Unidade em AuditorFiscal e como CodigoUnidade em UnidadeFiscal — são o mesmo domínio, mas com papéis (nomes) diferentes.

3.4 Domínio (Domain)

Um **domínio** é um conjunto de valores atômicos (indivisíveis) permitidos para um atributo. A cada domínio associa-se um tipo de dado ou formato.

Atributo	Domínio (descrição)	Tipo de dado em T-SQL
Matrícula	Números inteiros positivos de até 6 dígitos	INT
Salário	Valores decimais não negativos, com 2 casas decimais	NUMERIC(15,2)
Unidade	Um dos códigos de unidade fiscal cadastrados	CHAR(3) + CHECK/FOREIGN KEY
ChaveDeAcessoNFe	Sequência de exatamente 44 dígitos numéricos	CHAR(44) + CHECK

Domínio não é apenas tipo de dado: o domínio de **Salário** não é simplesmente "qualquer número decimal" — é o **conjunto de valores que fazem sentido no mundo real** para aquele atributo (por exemplo, nenhum salário é negativo). Um tipo de dado (**NUMERIC(15,2)**) é uma *aproximação técnica* do domínio; a constraint **CHECK (salario >= 0)**, estudada no módulo de Restrições e Integridade, é o que efetivamente impõe o domínio correto.

3.5 Esquema de Relação (Relation Schema)

Um **esquema de relação**, denotado por **R(A1, A2, ..., An)**, descreve a **estrutura** de uma relação **R** como um conjunto de atributos {**A1, A2, ..., An**}. É a "planta" da tabela — nome da relação e nomes dos atributos — **sem os dados propriamente ditos**.

Esquema: AuditorFiscal(**Matricula**, Nome, Endereco, Funcao, Salario, Unidade)

Por convenção, os atributos que compõem a **chave primária** são sublinhados (ou, em texto simples, destacados em **negrito**):

AuditorFiscal(**Matricula**, Nome, Endereco, Funcao, Salario, Unidade)

4. Esquema de Relação × Instância (Snapshot)

É essencial distinguir dois conceitos que, na prática do dia a dia, costumam ser confundidos:

Conceito	O que é	Estabilidade
Esquema de relação $R(A_1, \dots, A_n)$	A estrutura — nomes de atributos, domínios, chaves	Muda raramente (apenas por ALTER TABLE)
Instância (ou "snapshot") de relação $r(R)$	O conjunto de tuplas armazenadas em um dado momento	Muda a cada INSERT , UPDATE ou DELETE

Formalmente, uma instância $r(R)$ é um **subconjunto do produto cartesiano** dos domínios dos atributos de R :

$$r(R) \subseteq (\text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n))$$

Em outras palavras: uma instância é qualquer combinação válida de valores dos domínios dos atributos que, de fato, **existe naquele momento** na tabela.

Exemplo fiscal: o esquema `AutoInfracao(NumeroAuto, Contribuinte, TipoInfracao, ValorMultas, DataLavratura)` não muda com o tempo. Já a **instância** dessa relação — quais autos de infração realmente existem hoje na base — muda constantemente, à medida que novos autos são lavrados, anulados ou quitados. Uma consulta de "quantos autos de infração foram lavrados este mês" está, na prática, perguntando sobre a **instância atual**, não sobre o esquema.

5. Grau da Relação e Propriedades Formais

5.1 Grau (Degree)

O **grau** de uma relação é o número de atributos (colunas) do seu esquema.

A relação `AuditorFiscal(Matricula, Nome, Endereco, Funcao, Salario, Unidade)` tem **grau 6**.

Não confundir grau (número de colunas, uma propriedade do **esquema**, estável) com **cardinalidade da relação** (número de linhas em um dado momento, uma propriedade da **instância**, que muda o tempo todo).

5.2 Propriedades Formais de uma Relação

O Modelo Relacional, por derivar da teoria dos conjuntos, impõe quatro propriedades que **nem sempre são evidentes** para quem só conhece SQL na prática:

(a) A ordem das tuplas não tem significado. Matematicamente, uma relação é um **conjunto** de tuplas, e conjuntos não têm ordem inerente. Isso significa que, formalmente, não existe "primeira" ou "última" linha em uma relação — a ordem de exibição é responsabilidade da consulta (`ORDER BY`), não da relação em si.

Consequência prática: uma consulta `SELECT * FROM auto_infracao` sem `ORDER BY` **não tem garantia alguma de ordem** — mesmo que, na prática, o SQL Server costume devolver as linhas na ordem física de armazenamento. Para qualquer relatório fiscal onde a ordem importa (ex.: "os 10 maiores autos de infração"), é indispensável usar `ORDER BY` explicitamente.

(b) A ordem dos atributos não tem significado (no nível teórico). Assim como as tuplas, os atributos de um esquema formam um **conjunto**, não uma sequência. Na prática do SQL, colunas têm uma ordem física de armazenamento, mas semanticamente `SELECT Nome, Matricula FROM AuditorFiscal` traz a mesma informação que `SELECT Matricula, Nome FROM AuditorFiscal` — apenas em ordem diferente de exibição.

(c) Todo atributo tem valor atômico. Um valor de atributo, numa dada tupla, deve ser **um único valor indivisível** do domínio daquele atributo — nunca um conjunto de valores. Este é o princípio que fundamenta a **1ª Forma Normal**, e é exatamente a razão pela qual, no módulo anterior, a "Duplicata da NF-e" (que poderia ter vários valores para uma mesma nota) precisou virar uma tabela separada, em vez de uma única coluna com múltiplos valores concatenados.

(d) Todas as tuplas de uma relação devem ser únicas (não há tuplas duplicadas). Como uma relação é, formalmente, um **conjunto**, não pode haver duas tuplas

idênticas em todos os seus atributos.

Atenção — divergência importante entre teoria e prática do SQL: o SQL Server (como qualquer SGBD comercial) **não impõe automaticamente** essa regra. Uma tabela T-SQL sem **PRIMARY KEY** ou **UNIQUE** é, tecnicamente, um **multiconjunto (multiset ou "bag")**, que **permite linhas duplicadas**. Por exemplo, nada impede, por padrão, que a mesma combinação de **(Contribuinte, TipoInfraçao, DataLavratura, ValorMulta)** seja inserida duas vezes em **auto_infraçao**, gerando dois autos de infração "clones" — um erro operacional grave em um contexto fiscal. **É exatamente por isso que a definição de uma PRIMARY KEY (ou de constraints UNIQUE adequadas), tema do módulo de Restrições e Integridade, é o que aproxima uma tabela SQL real do conceito formal de "relação" no sentido matemático.**

6. Chaves de uma Relação

As definições de chave no Modelo Relacional seguem a mesma lógica dos identificadores discutidos no módulo de Modelagem Relacional Básica — aqui as revisitamos com mais formalismo, segundo a tradição de Codd/Elmasri.

6.1 Superchave (Superkey)

Uma **superchave** é qualquer subconjunto de atributos de uma relação que garante **unicidade** — ou seja, não existem (nem podem existir) duas tuplas distintas com os mesmos valores para todos os atributos da superchave.

Exemplo: em **AuditorFiscal**, o conjunto **{Matricula}** é uma superchave. O conjunto **{Matricula, Nome}** também é uma superchave (mas contém um atributo redundante, já que **Matricula** sozinha já garante a unicidade).

6.2 Chave Candidata (Candidate Key)

Uma **chave candidata** é uma superchave **mínima**: nenhum subconjunto próprio dela também é uma superchave.

Em **AuditorFiscal**, **{Matricula}** é uma chave candidata. Se o CPF do auditor também estivesse na tabela, **{CPF}** seria uma segunda chave candidata.

6.3 Chave Primária (Primary Key)

Dentre as chaves candidatas, uma é **escolhida** pelo projetista para ser a **chave primária** — o identificador oficial das tuplas daquela relação, e a que será referenciada por chaves estrangeiras de outras relações.

Por convenção, na notação de esquema, sublinha-se (ou destaca-se em negrito) o(s) atributo(s) que compõem a chave primária:

```
AuditorFiscal(**Matricula**, Nome, Endereco, Funcao, Salario, Unidade)
```

6.4 Chave Alternativa (Alternate Key)

Toda chave candidata que **não** foi escolhida como chave primária é chamada de **chave alternativa**. Ela continua sendo única, mas não é o identificador oficial nem é referenciada por chaves estrangeiras (a menos que explicitamente declarada com **UNIQUE**).

No exemplo do módulo anterior, **CNPJ/CPF** do contribuinte é uma chave alternativa (**UNIQUE**), enquanto **id_contribuinte** (surrogate key) foi escolhida como chave primária.

7. Chave Estrangeira e Autorrelacionamento

7.1 Definição Formal de Chave Estrangeira

Formalmente, um conjunto de atributos de uma relação **R1** é uma **chave estrangeira** se satisfaz **duas condições**:

1. Os atributos da chave estrangeira **compartilham o mesmo domínio** dos atributos que formam a chave primária de outra relação **R2** (ou da própria **R1**, no caso de autorrelacionamento).
2. Para toda tupla **t1** de **R1**, o valor da chave estrangeira em **t1** **ou** corresponde ao valor da chave primária de alguma tupla **t2** em **R2**, **ou** é **NULO**. Esta segunda

condição é exatamente a **integridade referencial** (seção 8.3).

7.2 Exemplo Fiscal Completo

Considere as relações `UnidadeFiscal` e `AuditorFiscal`, com a seguinte instância:

`UnidadeFiscal`

CodUnidade	Nome	MatriculaChefe
UF2	Regional Fiscalização Sul	210
UF1	Regional Fiscalização Norte	100
UF3	Regional Fiscalização Sertão	NULO

`AuditorFiscal`

Matrícula	Nome	Endereço	Função	Salário	Unidade
100	Ana Ribeiro	R. Pedro I, 12, Alto Branco	Auditor Tributário	12.500,00	UF1
250	Pedro Souza	R. J. Silva, 24, Liberdade	Auditor Fiscal Chefe	18.900,00	UF1
108	André Lima	R. Itália, 33, Bairro das Nações	Técnico Tributário	8.700,00	UF2
210	Paula Nogueira	R. Pará, 98, Bairro dos Estados	Auditora Fiscal Chefe	19.200,00	UF2

Aqui temos **duas** chaves estrangeiras envolvidas:

- `AuditorFiscal.Unidade` é uma chave estrangeira que referencia `UnidadeFiscal.CodUnidade` — todo auditor pertence a uma unidade fiscal existente.
- `UnidadeFiscal.MatriculaChefe` é uma chave estrangeira que referencia `AuditorFiscal.Matricula` — o chefe de uma unidade é ele próprio um auditor cadastrado. Note que a unidade `UF3` tem `MatriculaChefe = NULO`: pela definição formal (condição 2 acima), isso é **permitido** — significa apenas que

aquela unidade momentaneamente não tem chefe designado, e não viola a integridade referencial.

Em T-SQL:

```
CREATE TABLE unidade_fiscal (  
    cod_unidade      CHAR(3)      NOT NULL,  
    nome             VARCHAR(100) NOT NULL,  
    matricula_chefe  INT          NULL,      -- pode ser NULO  
    CONSTRAINT pk_unidade_fiscal PRIMARY KEY (cod_unidade)  
);  
  
CREATE TABLE auditor_fiscal (  
    matricula      INT          NOT NULL,  
    nome           VARCHAR(150) NOT NULL,  
    endereco       VARCHAR(200) NOT NULL,  
    funcao         VARCHAR(50)  NOT NULL,  
    salario        NUMERIC(15,2) NOT NULL,  
    cod_unidade    CHAR(3)      NOT NULL,  
    CONSTRAINT pk_auditor_fiscal PRIMARY KEY (matricula),  
    CONSTRAINT fk_auditor_unidade FOREIGN KEY (cod_unidade)  
        REFERENCES unidade_fiscal (cod_unidade)  
);  
  
-- Só é possível criar esta FK depois que 'auditor_fiscal' já existe,  
-- pois ela referencia 'auditor_fiscal.matricula':  
ALTER TABLE unidade_fiscal  
ADD CONSTRAINT fk_unidade_chefe FOREIGN KEY (matricula_chefe)  
    REFERENCES auditor_fiscal (matricula);
```

Observação de ordem de criação: este exemplo ilustra uma situação comum em modelagem — duas tabelas **referenciam-se mutuamente** (`auditor_fiscal` referencia `unidade_fiscal`, e `unidade_fiscal` referencia `auditor_fiscal`). Nesses casos, é preciso criar uma das FKs **depois** que ambas as tabelas já existirem, usando `ALTER TABLE ... ADD CONSTRAINT`, como feito acima.

7.3 Autorrelacionamento (Self-Referencing Foreign Key)

Uma chave estrangeira pode referenciar a **própria relação** a que pertence — é o chamado **autorrelacionamento**.

Exemplo clássico: `AuditorFiscal(Matricula, Nome, Salario, MatriculaSupervisor)`, em que `MatriculaSupervisor` referencia

AuditorFiscal.Matricula — ou seja, um auditor pode ter outro auditor como supervisor direto.

```
ALTER TABLE auditor_fiscal
ADD matricula_supervisor INT NULL;

ALTER TABLE auditor_fiscal
ADD CONSTRAINT fk_auditor_supervisor
FOREIGN KEY (matricula_supervisor) REFERENCES auditor_fiscal (matricula);
```

Este é o mesmo padrão já estudado no módulo de Modelagem Relacional Básica (Regra 6 de mapeamento) e no módulo de Regras de Mapeamento MER→Relacional (NF-e que referencia outra NF-e) — a chave estrangeira simplesmente aponta para a chave primária da mesma tabela em que está definida.

8. Regras de Integridade

O Modelo Relacional define três regras de integridade estruturais, que qualquer SGBD relacional — incluindo o SQL Server — deve ser capaz de garantir.

8.1 Integridade de Chave (Key Integrity)

Toda tupla de uma relação deve possuir um conjunto de atributos (a chave) que a identifica de forma única dentro daquela relação. Esta é a regra mais básica, subjacente à própria noção de chave candidata (seção 6.2).

8.2 Integridade de Entidade (Entity Integrity / Integridade de Identidade)

Nenhum atributo que faça parte da chave primária pode ter valor **NULO.**

Exemplo fiscal: seria impossível ter um **AutoInfracao** sem **NumeroAuto** (sua chave primária) — um auto de infração sem número não pode, por definição, ser identificado unicamente, o que inviabilizaria toda referência a ele (por exemplo, em um recurso administrativo).

Em T-SQL, esta regra é garantida automaticamente pela constraint **PRIMARY KEY**, que implicitamente torna a(s) coluna(s) **NOT NULL**.

8.3 Integridade Referencial (Referential Integrity)

Como já formalizado na seção 7.1: toda chave estrangeira deve, em cada tupla, **ou** corresponder a um valor de chave primária existente na relação referenciada, **ou** ser **NULA** (quando a FK permite **NULL**).

Exemplo fiscal: um **AutoInfracao.MatriculaAuditor** (chave estrangeira para **AuditorFiscal**) não pode conter uma matrícula que não exista em **AuditorFiscal** — isso representaria um auto de infração lavrado por um auditor "fantasma".

9. Operações de Atualização e Violações de Integridade

As três operações básicas que alteram uma instância de relação — **inserção**, **remoção** e **atualização (modificação)** — podem, cada uma, violar uma ou mais das regras de integridade vistas na seção 8. É importante entender **o que fazer diante de cada tipo de violação**, pois essa decisão de design tem impacto direto nos dados fiscais.

9.1 Inserção (INSERT)

Ao inserir uma nova tupla, podem ocorrer violações de:

- **Integridade de entidade**: tentar inserir uma tupla sem valor para a chave primária, ou com uma chave primária já existente.
- **Integridade referencial**: tentar inserir uma tupla cuja chave estrangeira aponta para um valor inexistente na relação referenciada.
- **Restrições de domínio** (**CHECK**, tipo de dado): tentar inserir um valor fora do domínio esperado do atributo.

Exemplo fiscal: tentar inserir o auto de infração ('A-2026-00931', 'Contribuinte X', 'Sonegação', -500.00, '2026-07-20', matricula_auditor=9999) — aqui há **duas** violações simultâneas: **ValorMulta** negativo (violação de domínio) e **matricula_auditor = 9999** inexistente (violação de integridade referencial).

O que fazer diante da violação numa inserção? Basicamente duas estratégias:

1. **Rejeitar a inserção** (comportamento padrão do SQL Server: a transação falha e nenhuma linha é inserida) — é a opção **recomendada** em bases fiscais, pois impede a entrada de dados inconsistentes desde a origem.
2. **Permitir a inserção "provisória" e corrigir depois** — abordagem mais arriscada, geralmente usada apenas em cargas em lote de sistemas legados, com posterior rotina de saneamento de dados. **Não recomendada** como prática permanente em bases de produção fiscal, pois abre uma janela de inconsistência.

9.2 Remoção (DELETE)

Uma remoção só pode violar a **integridade referencial** (nunca a integridade de entidade ou de domínio, pois remover uma tupla não introduz valores novos). O problema surge quando a tupla removida é **referenciada por chave estrangeira em outra relação**.

Exemplo fiscal: tentar excluir um **Contribuinte** que já possui **NotasFiscais** ou **AutosDeInfraacao** associados.

O que fazer diante da violação numa remoção? Três estratégias possíveis (já estudadas em profundidade no módulo de Restrições e Integridade, seção de **ON DELETE**):

1. **Rejeitar a remoção** (**NO ACTION**, padrão do SQL Server) — impede a exclusão do "pai" enquanto existirem "filhos". **Recomendado** para preservar o histórico fiscal (contribuintes, NF-e, autos de infração não devem desaparecer silenciosamente).
2. **Efeito cascata** (**CASCADE**) — remove automaticamente todas as tuplas que referenciam a tupla excluída. Adequado apenas para relações de "parte de um todo", como itens de um documento (**ItemNFe** em relação a **NFe**).
3. **Modificar a chave estrangeira referenciada para NULO** (**SET NULL**) — só é possível se a FK permitir **NULL** e não fizer parte da chave primária da tabela

"filha". Adequado quando a referência é opcional (ex.: transportadora de um CT-e).

9.3 Atualização (UPDATE)

Uma atualização pode violar qualquer uma das três regras de integridade, dependendo de qual atributo é alterado:

- Alterar a **chave primária** de uma tupla pode violar a integridade de entidade (se o novo valor já existir em outra tupla) e também exige decidir o que fazer com as chaves estrangeiras que a referenciavam em outras tabelas (**ON UPDATE CASCADE**, visto no módulo anterior).
- Alterar um **atributo de domínio restrito** (ex.: mudar **Situacao** de uma NF-e para um valor não previsto) viola a integridade de domínio, sendo barrada por um **CHECK**.
- Alterar a **chave estrangeira** para um valor inexistente na relação referenciada viola a integridade referencial, da mesma forma que na inserção.

Ligação direta com o módulo de Restrições e Integridade: a *trigger*

trg_impede_reabertura_nfe, estudada naquele módulo, é um exemplo prático de como impedir uma atualização especificamente inválida (mudar a situação de uma NF-e de **CANCELADA** para **AUTORIZADA**) — uma regra de negócio que vai além das três regras de integridade estruturais, mas segue a mesma lógica de "avaliar a operação de atualização antes de permiti-la".

10. Álgebra Relacional — Fundamentos e Operadores Básicos

10.1 O que é a Álgebra Relacional

A **álgebra relacional** é uma linguagem de consulta **procedural** e **formal**: cada operador recebe uma ou duas relações como entrada (operandos) e produz **uma nova relação** como resultado. Essa propriedade — o resultado de qualquer operação ser, ele próprio, uma relação — chama-se **fechamento (closure)**, e é o que permite **compor** operadores em expressões mais complexas (o resultado de uma operação vira o operando de outra).

A álgebra relacional é o fundamento teórico sobre o qual o SQL foi construído: toda consulta SQL, por mais complexa que pareça, pode ser reduzida a uma combinação dos operadores que veremos a seguir. Compreendê-la profundamente ajuda a **entender e otimizar** consultas SQL do dia a dia, além de facilitar a comunicação técnica sobre o que uma consulta efetivamente faz.

Cinco operadores são considerados **fundamentais** — todos os demais podem ser expressos a partir deles (a interseção, a junção e a divisão, vistas na seção 11, são chamadas de **operadores derivados** por esse motivo).

10.2 Seleção (Selection) — σ

Seleciona um subconjunto das tuplas de uma relação que satisfazem uma condição lógica (predicado), mantendo todos os atributos.

σ condição (Relação)

Exemplo fiscal: selecionar todos os autos de infração com valor de multa superior a R\$ 10.000,00.

σ ValorMulta > 10000 (AutoInfracao)

Em T-SQL, a seleção corresponde à cláusula **WHERE**:

```
SELECT *  
FROM auto_infracao  
WHERE valor_multa > 10000;
```

10.3 Projeção (Projection) — π

Copia a relação dada, mas mantendo apenas os **atributos especificados**, descartando os demais. Como o resultado é uma relação (um conjunto), **linhas duplicadas resultantes são eliminadas**.

π atributo1, atributo2 (Relação)

Exemplo fiscal: obter apenas a matrícula e o nome de todos os auditores fiscais, sem repetir combinações idênticas.

```
 $\pi$  Matricula, Nome (AuditorFiscal)
```

Em T-SQL:

```
SELECT DISTINCT matricula, nome  
FROM auditor_fiscal;
```

Nuance importante: em álgebra relacional pura, a projeção **sempre** remove duplicatas (por definição de relação como conjunto — seção 5.2-d). Em SQL, por padrão, **SELECT** **não** remove duplicatas automaticamente — é preciso usar **DISTINCT** explicitamente para reproduzir fielmente a semântica da projeção formal. Essa é uma das diferenças mais citadas entre a álgebra relacional teórica e o SQL prático.

10.4 Produto Cartesiano (Cartesian Product) — $\times$

Combina **cada tupla** de uma relação com **cada tupla** de outra, produzindo todas as combinações possíveis. Se **R1** tem **m** tuplas e **R2** tem **n** tuplas, o resultado **R1 $\times$ R2** terá **m $\times$ n** tuplas.

```
AuditorFiscal  $\times$  UnidadeFiscal
```

Exemplo fiscal: combinar cada auditor com cada unidade fiscal — mesmo que o auditor não pertença àquela unidade. Sozinho, o produto cartesiano quase nunca é útil para responder a uma pergunta de negócio real (a maioria das combinações geradas não faz sentido); ele serve principalmente como **base** para a operação de junção (seção 11.2), que filtra apenas as combinações relevantes.

Em T-SQL, o produto cartesiano corresponde a um **CROSS JOIN**:

```
SELECT *  
FROM auditor_fiscal
```

```
CROSS JOIN unidade_fiscal;
```

Cuidado prático: um **CROSS JOIN** (ou uma consulta com múltiplas tabelas no **FROM** sem condição de junção) entre tabelas fiscais grandes — por exemplo, **Contribuinte** × **NotaFiscal** — pode gerar **bilhões de linhas** e travar o servidor. É importante que os profissionais da SEFAZ-PB reconheçam esse padrão de erro (esquecer a condição de junção) como uma das causas mais comuns de consultas SQL que "nunca terminam".

10.5 União (Union) — $\cup$

Combina o conjunto de tuplas de duas relações em uma única relação, **eliminando duplicatas**. Exige que as duas relações tenham o **mesmo esquema** (mesmo número e tipo de atributos, com significado compatível) — condição chamada de **compatibilidade de união (union compatibility)**.

```
ContribuintesEmMalhaFiscal  $\cup$  ContribuintesInadimplentes
```

Exemplo fiscal: obter o conjunto de todos os contribuintes que estão **ou** na malha fiscal (por inconsistências de cruzamento NF-e × EFD), **ou** inadimplentes (débitos em aberto), sem repetir contribuintes que se enquadrem em ambas as situações.

Em T-SQL:

```
SELECT id_contribuinte FROM contribuinte_em_malha_fiscal
UNION
SELECT id_contribuinte FROM contribuinte_inadimplente;
```

Assim como a projeção, a **UNION** do T-SQL já remove duplicatas por padrão (correspondendo fielmente à álgebra relacional). Se a eliminação de duplicatas **não** for desejada — por exemplo, para contar quantas vezes um contribuinte aparece em cada critério —, usa-se **UNION ALL**, que **não** tem equivalente direto na álgebra relacional pura (pois um multiconjunto não é uma relação, no sentido formal).

10.6 Diferença entre Conjuntos (Set Difference) — —

Produz as tuplas que existem na **primeira** relação, mas **não** existem na segunda. Assim como a união, exige compatibilidade de união entre as duas relações.

TodosOsContribuintesAtivos - ContribuintesComEFDEntregue

Exemplo fiscal: obter todos os contribuintes ativos que **ainda não** entregaram a EFD do período — um indício direto e imediato de possível omissão de escrituração.

Em T-SQL, a diferença de conjuntos corresponde ao operador **EXCEPT**:

```
SELECT id_contribuinte FROM contribuinte WHERE situacao_cadastral = 'ATIVO'
EXCEPT
SELECT id_contribuinte FROM efd_arquivo WHERE periodo_apuracao = '07/2026';
```

Atenção à ordem dos operandos: ao contrário da união (que é comutativa: $A \cup B = B \cup A$), a diferença **não é comutativa** ($A - B \neq B - A$). Trocar a ordem no **EXCEPT** muda completamente o significado da consulta — de "quem não entregou" para "quem entregou, mas por algum motivo não está mais ativo".

11. Operadores Derivados: Junção, Interseção e Divisão

Embora os cinco operadores da seção 10 sejam formalmente suficientes para expressar qualquer consulta relacional, três operadores adicionais — derivados a partir dos fundamentais — são tão utilizados na prática que merecem tratamento próprio.

11.1 Interseção (Intersection) — $\cap$

Produz as tuplas que existem **em ambas** as relações. Pode ser expressa a partir da diferença: $A \cap B = A - (A - B)$. Exige compatibilidade de união.

Exemplo fiscal: contribuintes que estão **simultaneamente** na malha fiscal e inadimplentes — um grupo de prioridade máxima para ação de cobrança e fiscalização conjunta.

Em T-SQL, a interseção corresponde ao operador **INTERSECT**:

```
SELECT id_contribuinte FROM contribuinte_em_malha_fiscal
INTERSECT
SELECT id_contribuinte FROM contribuinte_inadimplente;
```

11.2 Junção (Join) — ⋈

A **junção** é, na prática, o operador mais usado no dia a dia de consultas fiscais. Ela combina tuplas de duas relações que satisfazem uma condição sobre atributos comuns — formalmente, pode ser expressa como um **produto cartesiano seguido de uma seleção**:

$$R1 \bowtie \text{condição } R2 \quad \equiv \quad \sigma \text{ condição } (R1 \times R2)$$

Um caso particular muito comum é a **junção natural (natural join)**, em que a condição de junção é a igualdade entre atributos de mesmo nome (tipicamente, uma chave estrangeira e a chave primária que ela referencia), e a coluna duplicada é eliminada do resultado.

Exemplo fiscal: obter o nome do auditor responsável junto com os dados de cada auto de infração que ele lavrou.

```
AutoInfracao ⋈ AutoInfracao.MatriculaAuditor = AuditorFiscal.Matricula
AuditorFiscal
```

Em T-SQL, a junção corresponde ao **INNER JOIN** (ou, na sintaxe mais antiga, a listar as tabelas no **FROM** separadas por vírgula, com a condição no **WHERE** — desaconselhada, pois mistura visualmente a junção com filtros de seleção comuns):

```
-- Forma recomendada (deixa explícita a diferença entre condição de junção e
filtro)
SELECT a.numero_auto, a.valor_multa, af.nome AS nome_auditor
FROM auto_infracao a
```

```
INNER JOIN auditor_fiscal af ON a.matricula_auditor = af.matricula
WHERE a.valor_multa > 10000;
```

Repare como esta consulta única já combina, na prática, **três** operadores da álgebra relacional: uma **junção** (`INNER JOIN ... ON`), uma **seleção** (`WHERE a.valor_multa > 10000`) e uma **projeção** (a lista de colunas do `SELECT`). Isso ilustra bem por que a álgebra relacional é útil: qualquer consulta SQL, por mais complexa, pode ser decomposta nesses blocos fundamentais para fins de entendimento, depuração e otimização.

11.3 Divisão (Division) — ÷

A **divisão** é o operador mais elaborado (e menos intuitivo) da álgebra relacional, mas resolve um tipo específico de pergunta muito comum em fiscalização: **"quem satisfaz TODAS as condições de um determinado conjunto?"**

Exemplo fiscal: identificar quais **auditores fiscais** participaram de **absolutamente todas** as unidades fiscais existentes ao longo de sua carreira (por exemplo, para fins de elegibilidade a um cargo que exige rodízio completo entre regionais).

Formalmente, dadas duas relações $R(A, B)$ e $S(B)$, a divisão $R \div S$ produz os valores de A que aparecem em R associados a **todos** os valores de B presentes em S .

Não existe uma cláusula única `DIVIDE` em T-SQL — a divisão é tipicamente implementada combinando `GROUP BY`, `HAVING` e uma subconsulta de contagem:

```
-- Auditores que já atuaram em TODAS as unidades fiscais cadastradas
SELECT h.matricula_auditor
FROM historico_lotacao_auditor h
GROUP BY h.matricula_auditor
HAVING COUNT(DISTINCT h.cod_unidade) = (SELECT COUNT(*) FROM unidade_fiscal);
```

A lógica aqui é: para cada auditor, contamos **quantas unidades distintas** ele já integrou (`COUNT(DISTINCT h.cod_unidade)`), e comparamos com o **total de unidades fiscais existentes** (`SELECT COUNT(*) FROM unidade_fiscal`). Se os dois números coincidirem, aquele auditor "cobriu" todas as unidades — exatamente o resultado que a divisão relacional formal produziria.

12. Roteiro Prático — Do Conceito Formal ao T-SQL

Objetivo: implementar as relações `UnidadeFiscal` e `AuditorFiscal` (seção 7.2) no SSMS e reproduzir, em T-SQL, cada operador da álgebra relacional estudado.

1. No SSMS, execute o script de criação de tabelas da seção 7.2 no banco `curso_integridade_fiscal`.
2. Insira as tuplas de exemplo apresentadas na seção 7.2 (atenção à ordem de inserção: como as tabelas se referenciam mutuamente, pode ser necessário inserir primeiro os auditores sem `matricula_supervisor/unidade_fiscal.matricula_chefe`, e depois usar `UPDATE` para completar essas referências).
3. Para cada expressão de álgebra relacional abaixo, escreva e execute o comando T-SQL equivalente, e confira o resultado:

#	Expressão em álgebra relacional (informal)	O que fazer no SSMS
a	σ Salario > 15000 (AuditorFiscal)	Escrever o <code>SELECT ... WHERE</code> equivalente
b	π Nome, Unidade (AuditorFiscal)	Escrever o <code>SELECT DISTINCT</code> equivalente
c	$\text{AuditorFiscal} \bowtie \text{Unidade} = \text{CodUnidade} \text{ UnidadeFiscal}$	Escrever o <code>INNER JOIN</code> equivalente, trazendo <code>auditor.nome</code> e <code>unidade.nome</code>
d	Auditores da UF1 $\cup$ Auditores da UF2	Escrever a consulta com <code>UNION</code>
e	Auditores da UF1 – Auditores com Salario > 15000	Escrever a consulta com <code>EXCEPT</code>

4. Discuta em grupo: para qual dessas expressões o `DISTINCT`/eliminação de duplicatas fez diferença real no resultado, e para qual não fez (por quê)?

13. Exercícios Práticos

Exercício 1 — Identificando os conceitos (teórico)

Com base na relação `AutoInfracao(NumeroAuto, Contribuinte, TipoInfracao, ValorMulta, DataLavratura, MatriculaAuditor)`: a) Qual é o grau desta relação? b) Aponte um exemplo de superchave que não seja uma chave candidata. c) Qual atributo você recomendaria como chave primária, e por quê? d) Qual atributo é uma chave estrangeira, e para qual relação ele aponta?

Exercício 2 — Esquema × Instância

Explique, com suas próprias palavras e um exemplo fiscal, por que a afirmação "a tabela `NotaFiscal` tem 3 milhões de linhas" é uma afirmação sobre a **instância**, e não sobre o **esquema**, da relação `NotaFiscal`.

Exercício 3 — Diagnosticando violações de integridade

Para cada operação abaixo, indique **qual regra de integridade** seria violada (chave, entidade/identidade ou referencial), ou se nenhuma seria violada:

a) `INSERT` em `AutoInfracao` com `NumeroAuto = NULL`. b) `INSERT` em `AutoInfracao` com `MatriculaAuditor = 9999` (matrícula inexistente). c) `DELETE` em `Contribuinte` de um contribuinte que já possui NF-e emitidas. d) `UPDATE` em `AuditorFiscal` mudando a `Matricula` de um auditor que é referenciado por vários autos de infração já lavrados, sem `ON UPDATE CASCADE`.

Exercício 4 — Traduzindo álgebra relacional para T-SQL

Traduza cada expressão para uma consulta T-SQL equivalente, assumindo o esquema da seção 7.2 (mais `AutoInfracao`, já estudada em módulos anteriores):

a) $\pi_{\text{Nome}} (\sigma_{\text{Salario} > 15000} (\text{AuditorFiscal}))$ b) $\sigma_{\text{TipoInfracao} = \text{'Sonegação'}} (\text{AutoInfracao}) \bowtie \text{MatriculaAuditor} = \text{Matricula AuditorFiscal}$

c) π Contribuinte (AutoInfracao) - π id_contribuinte
(ContribuinteInadimplente)

Exercício 5 — Divisão relacional na prática

Considerando a tabela `historico_lotacao_auditor(matricula_auditor, cod_unidade, data_inicio, data_fim)`, escreva uma consulta T-SQL que identifique quais auditores **nunca** atuaram na unidade 'UF3' — este é o **complemento** da divisão relacional (quem NÃO satisfaz uma condição para determinado valor específico). Compare com a consulta de divisão apresentada na seção 11.3 e explique a diferença de abordagem.

Exercício 6 (Desafio) — Relação como conjunto vs. tabela SQL

A tabela `auto_infracao`, no SQL Server, foi criada **sem** nenhuma **PRIMARY KEY** ou **UNIQUE** (um erro de modelagem). Um analista insere, por engano, o mesmo auto de infração duas vezes, com todos os atributos idênticos. a) Isso seria possível em uma relação, no sentido formal do Modelo Relacional? Justifique com base na seção 5.2(d). b) O SQL Server, por padrão, aceitaria essa segunda inserção? Por quê? c) Escreva o **ALTER TABLE** que corrige essa tabela, adicionando uma **PRIMARY KEY** adequada.

14. Glossário

Termo	Definição
Relação	Estrutura formal equivalente a uma tabela; um conjunto de tuplas com os mesmos atributos
Tupla	Uma linha de uma relação
Atributo	Uma coluna de uma relação; um papel desempenhado por um domínio
Domínio	Conjunto de valores atômicos válidos para um atributo
Esquema de relação	A estrutura (nomes de atributos e domínios) de uma relação, denotada $R(A_1, \dots, A_n)$

Termo	Definição
Instância (snapshot)	O conjunto de tuplas armazenadas em uma relação em um dado momento
Grau	Número de atributos de uma relação
Cardinalidade da relação	Número de tuplas de uma relação em um dado momento
Superchave	Conjunto de atributos que garante unicidade de tuplas (pode ter atributos redundantes)
Chave candidata	Superchave mínima
Chave primária	Chave candidata escolhida como identificador oficial da relação
Chave alternativa	Chave candidata não escolhida como primária
Chave estrangeira	Atributo(s) que referenciam a chave primária de outra relação (ou da mesma, em autorrelacionamento)
Integridade de chave	Toda tupla deve ser identificável de forma única
Integridade de entidade	Nenhum atributo da chave primária pode ser nulo
Integridade referencial	Toda chave estrangeira deve referenciar um valor existente, ou ser nula
Álgebra relacional	Linguagem formal e procedural de consulta, cujos operadores produzem relações a partir de relações
Fechamento (closure)	Propriedade pela qual o resultado de qualquer operação da álgebra relacional é, ele próprio, uma relação
Seleção (σ)	Filtra tuplas de uma relação segundo uma condição
Projeção (π)	Mantém apenas os atributos especificados, eliminando duplicatas
Produto cartesiano ($\times$)	Combina todas as tuplas de duas relações entre si
União ($\cup$)	Combina as tuplas de duas relações compatíveis, eliminando duplicatas

Termo	Definição
Diferença ($-$)	Tuplas presentes na primeira relação, mas ausentes na segunda
Interseção ($\cap$)	Tuplas presentes em ambas as relações
Junção ($\bowtie$)	Produto cartesiano seguido de seleção sobre atributos relacionados
Divisão ($\div$)	Identifica valores associados a TODOS os valores de outro conjunto

15. Referências

- CODD, E. F. "A Relational Model of Data for Large Shared Data Banks". *Communications of the ACM*, 1970.
- ELMASRI, R.; NAVATHE, S. *Fundamentals of Database Systems* (edição em português: *Sistemas de Banco de Dados*). 7ª ed. Pearson.
- DATE, C. J. *Introdução a Sistemas de Banco de Dados*. 8ª ed. Campus.
- HEUSER, C. A. *Projeto de Banco de Dados*. 6ª ed. Bookman.
- Documentação oficial da Microsoft — "SELECT (Transact-SQL)":
<https://learn.microsoft.com/sql/t-sql/queries/select-transact-sql>
- Documentação oficial da Microsoft — "Set Operators (EXCEPT, INTERSECT)":
<https://learn.microsoft.com/sql/t-sql/language-elements/set-operators-except-and-intersect>

Material elaborado para uso interno no curso de Banco de Dados Relacionais — Módulo O Modelo Relacional — Conceitos Fundamentais — SEFAZ-PB.